

Partie 1 : Questions de cours)

1. Qu'est-ce qu'un routeur dans Laravel et quelle est sa fonction principale ?
 2. Quelle est la différence entre une migration et un seeder ?
 3. Expliquez le principe de la validation des données dans un contrôleur Laravel.
 4. À quoi sert le middleware dans Laravel ?
 5. Donnez la commande Artisan pour créer un modèle avec une migration.
 6. Quelle est la structure d'un contrôleur de ressource dans Laravel?
 7. Expliquez le rôle d'Eloquent ORM dans Laravel.
 8. Donnez un exemple de relation "One to Many" entre deux modèles avec leurs fonctions associées.
 9. Quelle est la différence entre GET et POST dans les routes ?
 10. Citez deux méthodes pour protéger un formulaire contre les attaques CSRF dans Laravel.
-

Partie 2 : Exercices pratiques

Exercice 1 :

Création du formulaire d'inscription (4 points)

Objectif :

1- Créer une vue Blade avec le formulaire suivant :

Champs : Prénom & Nom, Date de naissance, Email, Téléphone, Mot de passe

2-Ajoutez une validation côté serveur pour s'assurer que :

- Tous les champs sont obligatoires
- Email est unique et bien formé
- Le mot de passe contient au moins 8 caractères

Tâches attendues :

Route GET /etudiants/create

Vue Blade etudiants/create.blade.php

Contrôleur EtudiantController avec la méthode store()

Redirection avec message de succès en cas de validation

Exercice2 :

La base de données est la suivante : Clients (CodeCli, nom, prénom, Ville, date naissance, adresse) ListeArticle(CodeArt, #CodeCli , NomArticle, PU, Qt)

1. créer une nouvelle application laravel nommé biblio
2. créer les migrations et modèles nécessaires pour les tables et exécuter les migrations (ajouter les relations entre les tables)
3. créer un Controller ListeArticleController
4. créer les routes l'application dans le fichier routes/web .les routes inclure des routes pour créer, afficher ; mettre à jour et supprimer des Articles
5. ajouter au controller une méthode afficherArticle et une view qui permet afficher la liste des Articles
6. ajouter une méthode ajouterArticle et une view qui permet ajouter un nouveau Article
7. ajouter une méthode supprimerArticle qui permet de supprimer un Article dans la table

8. Créer un middleware qui permet de vérifier existences des clients et interdit accès a url http d'une route /ListeArticle et rediriger vers les routes /client

Voila la correction :

1. Créer une application Laravel nommée biblio

laravel new biblio

cd biblio

2. Créer les migrations et les modèles nécessaires

a) Modèle et migration pour Client

php artisan make:model Client -m

Migration :

Schema::create('clients', function (Blueprint \$table) {

```
$table->id('CodeCli');  
$table->string('nom');  
$table->string('prenom');  
$table->string('ville');  
$table->date('date_naissance');  
$table->string('adresse');  
$table->timestamps();  
});
```

b) Modèle et migration pour ListeArticle

php artisan make:model ListeArticle -m

Migration :

```
Schema::create('liste_articles', function (Blueprint $table) {  
    $table->id('CodeArt');  
    $table->unsignedBigInteger('CodeCli');  
    $table->string('NomArticle');  
    $table->decimal('PU', 8, 2);  
    $table->integer('Qt');  
    $table->timestamps();  
});
```

```
$table->foreign('CodeCli')->references('CodeCli')-  
>on('clients')>onDelete('cascade');  
});
```

c) Relations dans les modèles

Client.php

```
public function articles()  
{  
    return $this->hasMany(ListeArticle::class, 'CodeCli');  
}
```

ListeArticle.php

```
public function client()  
{  
    return $this->belongsTo(Client::class, 'CodeCli');  
}
```

d) Exécuter les migrations

php artisan migrate

3. Créer un contrôleur ListeArticleController

php artisan make:controller ListeArticleController

4. Créer les routes dans routes/web.php

use App\Http\Controllers\ListeArticleController;

**Route::get('/articles', [ListeArticleController::class,
'afficherArticle']->name('articles.index'));**

**Route::get('/articles/create', [ListeArticleController::class,
'ajouterArticle']->name('articles.create'));**

**Route::post('/articles', [ListeArticleController::class,
'store'])>name('articles.store');**

**Route::delete('/articles/{CodeArt}', [ListeArticleController::class,
'supprimerArticle']->name('articles.destroy'));**

5. Méthode afficherArticle + Vue articles/index.blade.php

Controller :

```
public function afficherArticle()
{
    $articles = ListeArticle::with('client')->get();
    return view('articles.index', compact('articles'));
}
```

Vue : resources/views/articles/index.blade.php (extrait)

```
@foreach($articles as $article)
    <p>{{ $article->NomArticle }} - {{ $article->client->nom }}</p>
@endforeach
```

6. Méthode ajouterArticle + Vue articles/create.blade.php

Controller :

```
public function ajouterArticle()
{
    $clients = Client::all();
    return view('articles.create', compact('clients'));
}
```

```
public function store(Request $request)
{
    $request->validate([
        'CodeCli' => 'required|exists:clients,CodeCli',
        'NomArticle' => 'required',
        'PU' => 'required|numeric',
        'Qt' => 'required|integer',
    ]);

    ListeArticle::create($request->all());
    return redirect()->route('articles.index');
}
```

Vue : resources/views/articles/create.blade.php (extrait)

```
<form method="POST" action="{{ route('articles.store') }}">
    @csrf
    <select name="CodeCli">
```

```
@foreach($clients as $client)
    <option value="{{ $client->CodeCli }}">{{ $client->nom
}}</option>
@endforeach
</select>
<!-- Autres champs NomArticle, PU, Qt -->
<button type="submit">Ajouter</button>
</form>
```

7. Méthode supprimerArticle

Controller :

```
public function supprimerArticle($CodeArt)
{
    ListeArticle::findOrFail($CodeArt)->delete();
    return redirect()->route('articles.index');
}
```

8. Créer un Middleware CheckClientsExist

php artisan make:middleware CheckClientsExist

Middleware :

```
public function handle($request, Closure $next)
{
    if (\App\Models\Client::count() == 0) {
        return redirect('/clients')->with('error', 'Veuillez d\'abord saisir
des clients.');
```



```
    }

    return $next($request);
}
```

Enregistrement dans app/Http/Kernel.php :

```
'check.clients' => \App\Http\Middleware\CheckClientsExist::class,
```

Application dans routes :

```
Route::middleware('check.clients')->group(function () {
```

```
Route::get('/articles', [ListeArticleController::class,  
'afficherArticle']);  
});
```